

Modern OpenGL

동명대학교 게임공학과

강영민

현대적 오픈지엘

- OpenGL 3.3 Core Profile (2009년경) 이후 완전 변화
- 고정 기능 파이프라인(Fixed Function Pipeline) 퇴출
- 프로그래머가 100% 직접 제어하는 현대식 GPU 프로그래밍

구분	옛날 OpenGL (1.x~2.1)	현대적 OpenGL (3.3 Core ~ 4.6, 2025년 기준)
파이프라인 방식	고정 기능 파이프라인 glBegin/glEnd, glLight, glMaterial 등	완전 프로그래머블 파이프라인 Vertex Shader → Tessellation → Geometry → Fragment Shader → Compute Shader
셰이더 필수 여부	선택 (없어도 됨)	100% 필수! 셰이더 없이 삼각형 하나도 못 그림
상태 머신	거대한 전역 상태 머신 수천 개 glEnable/glDisable	상태 머신 거의 사라짐 → VAO + Uniform + Texture binding으로 명시적 제어
버퍼 관리	클라이언트 배열, 즉시 모드	VBO(Vertex Buffer Object), UBO, SSBO, Indirect Draw Buffer 등 필수
디버깅	거의 불가능	RenderDoc, NVIDIA Nsight, AMD Radeon GPU Profiler, apitrace 등으로 프레임 단위 디버깅 가능
확장성	제한적	AZDO(Approaching Zero Driver Overhead), Multi-Draw Indirect, Bindless Texture, Sparse Texture, Mesh Shader(확장) 등 극한 최적화 가능
플랫폼 지원	거의 모든 옛날 하드웨어	Core Profile은 OpenGL 3.3+ 지원 GPU만 (2009년 이후 GPU면 거의 다 지원됨)

주요 변화

GPU를 완전히 프로그래머가 제어하는, 셰이더 중심의 저수준 그래픽스 API

- glBegin/glEnd, glRotate, glLight 같은 건 15년째 deprecated
- 이제는 Vulkan/Metal/DirectX 12와 거의 같은 수준의 제어권을 제공

그러나 코딩은 더욱 복잡하고 어려워짐

- 대신 성능은 극한까지 뽑을 수 있음 (AZDO 기법 적용 시 드라이버 오버헤드 거의 0)

2025년 현재, 게임 엔진(Unity 2024+, Unreal Engine 5.4+, Godot 4.3+)

- 모두 현대 OpenGL Core Profile 또는 Vulkan을 사용

코딩의 난이도

항목	옛날 OpenGL (1.1 ~ 2.1, 2000년대 초중반)	현대 OpenGL (3.3 Core ~ 4.6, 2025년 기준)
삼각형 하나 띄우는 코드	약 15줄 (glBegin~glEnd)	최소 120~200줄 (GLFW + GLEW + VAO + VBO + 셰이더 컴파일 + 링크 + 유니폼 설정 등)
초보자 입문 난이도	★☆☆☆☆ (대학교 1학년도 가능)	★★★★★ (대학원 레벨)
필요한 개념 수	10개 이내 (glColor, glVertex, glTranslate 등)	50개 이상 (VAO, VBO, UBO, SSBO, FBO, GLSL, Uniform, Sampler, Binding point, Debug output, AZDO 등)
디버깅 도구	거의 없음 → 눈으로 때려맞춤	RenderDoc, Nsight 있긴 하지만 여전히 지옥 셰이더 잘못되면 검은 화면만 나옴
첫 결과 나오는 시간	10분	초보자 기준 3일~2주 (셰이더 컴파일 오류만 잡아도 하루 감)
실무에서 쓰는 사람	게임 회사 신입도 바로 사용	거의 없음 (Unreal/Unity/Vulkan/Metal 등을 사용)

Core Profile이 뭔가

이름	언제 나왔나?	어떤 함수를 쓸 수 있나?	비유
Compatibility Profile	옛날부터 계속	glBegin, glEnd, glRotatef, glLight, glMaterial 등 수 많은 함수를 지원	넓은 가구 500개
Core Profile	2008년 OpenGL 3.2부터	옛날 함수 99% 삭제 → 셰이더, VAO, VBO만 남김 → 사라진 함수의 기능을 프로그래머의 몫	새 아파트: 냉장고·에어컨·인덕션 고풍스런 장식장은 버리자

버린 대표 함수들 (Compatibility에서만 됨)

glBegin() / glEnd()

glTranslatef(), glRotatef(), glScalef()

glEnable(GL_LIGHTING), glLightfv()

glMatrixMode(), glLoadIdentity()

glVertex3f(), glColor3f()

Core Profile에서는 어떻게 해야 하나?

→ 절대 못 씸. 무조건 VBO + VAO + glDrawArrays

→ 행렬 직접 계산하거나 glm 같은 라이브러리 써야 함

→ 조명도 프래그먼트 셰이더에서 직접 계산

→ 유니폼으로 직접 행렬 전달 (camera.apply(shader)처럼)

→ 정점 버퍼에 미리 넣고 셰이더에서 읽음

기본 코딩 – 시작 프로그램 (1/10)

코딩의 예시 – glfw (myglfw.py)는 그대로 사용 가능

```
# main.py
import glfw
from OpenGL.GL import *
from OpenGL.GL.shaders import compileProgram, compileShader
import numpy as np
from Camera import Camera
from myglfw import *
```

현대 OpenGL 코딩에서 가장 중요한 “쉐이더 컴파일 헬퍼 함수” 두 개를 불러오는 것

기본 코딩 – 시작 프로그램 (2/10)

정점을 정의해 보자

```
# 정점 데이터: 위치(x,y,z) + 색상(r,g,b)
vertices = np.array([
    # 위치      # 색상
    1.0, 0.0, 0.0, 1.0, 0.0, 0.0, # X축 끝 / 빨강
    0.0, 1.0, 0.0, 0.0, 1.0, 0.0, # Y축 끝 / 초록
    0.0, 0.0, 1.0, 0.0, 0.0, 1.0, # Z축 끝 / 파랑
], dtype=np.float32)
```

정점 데이터에 위치와 색상을 담고 있음

기본 코딩 – 시작 프로그램 (3/10)

정점 셰이더 구현

셰이더 (정적 카메라, 회전 없음)

```
vs = ""
```

```
#version 460 core
```

```
layout(location=0) in vec3 aPos;
```

```
layout(location=1) in vec3 aColor;
```

```
out vec3 vColor;
```

```
uniform mat4 view;
```

```
uniform mat4 proj;
```

```
void main()
```

```
{
```

```
    gl_Position = proj * view * vec4(aPos, 1.0);
```

```
    vColor = aColor;
```

```
}
```

```
""
```

각 정점에 사용되는 셰이더 코드

- 정점의 위치
- 정점의 색상

두 정보를 받아서 출력으로 연결

- 위치는 뷰행렬이 곱해지고, 투영행렬 곱함
- 색상은 입력된 색상 그대로 출력

이 정보는 프래그먼트 셰이더로 전달

기본 코딩 – 시작 프로그램 (4/10)

프래그먼트(픽셀) 셰이더 구현

```
#version 460 core
```

```
in vec3 vColor;    → 버텍스 셰이더가 넘겨준 색상
```

```
out vec4 fragColor; → 이 픽셀의 최종 색상 (RGBA)
```

```
void main() {
```

```
    fragColor = vec4(vColor, 1.0);
```

```
    // → 넘겨받은 RGB 색상에 투명도 1.0(완전 불투명)을 붙여서 화면에 칠한다
```

```
}
```

정점 셰이더는 정점의 정보를 보간하여 프래그먼트 셰이더로 넘김

- 각 정점의 색이 보간되어 프래그먼트에 전달
- 그 값을 그대로 출력하는 콘

기본 코딩 – 시작 프로그램 (5/10)

쉐이더 ID와 VAO ID 저장 변수 선언

```
# 전역 변수  
shader = None  
vao = None
```

쉐이더를 컴파일해서 전달

```
def init_gl(window):  
    global shader, vao
```

```
# 쉐이더 컴파일  
shader = compileProgram(  
    compileShader(vs, GL_VERTEX_SHADER),  
    compileShader(fs, GL_FRAGMENT_SHADER)  
)
```

기본 코딩 – 시작 프로그램 (6/10)

VAO/VBO 준비

`vao = glGenVertexArrays(1)` # 1단계: 빈 상자 하나 만들어줘

`vbo = glGenBuffers(1)` # 2단계: 빈 서랍 하나 만들어줘

`glBindVertexArray(vao)` # 3단계: "지금부터 이 상자 안에 넣어!"

`glBindBuffer(GL_ARRAY_BUFFER, vbo)` # 4단계: "그리고 데이터는 이 서랍에!"

`glBufferData(GL_ARRAY_BUFFER, vertices.nbytes, vertices, GL_STATIC_DRAW)`

5단계: "자, 여기 정점 데이터 72바이트 담아!"

기본 코딩 – 시작 프로그램 (7/10)

VAO – 정점 데이터를 어떻게 해석하고 어디서 가져올지 모든 설정을 한 번에 기억해 두는 상태 객체

VBO – 정점 데이터 그 자체를 GPU 메모리에 보관하는 컨테이너

코드 줄	실제로 일어나는 일 (GPU 내부)
<code>vao = glGenVertexArrays(1)</code>	GPU가 빈 VAO를 하나 만들고 ID(예: 5번)를 돌려줌
<code>vbo = glGenBuffers(1)</code>	GPU가 빈 VBO를 하나 만들고 ID(예: 9번)를 돌려줌
<code>glBindVertexArray(vao)</code>	지금부터 모든 정점 설정이 5번 VAO에 저장됨
<code>glBindBuffer(GL_ARRAY_BUFFER, vbo)</code>	지금부터 버퍼 명령은 9번 VBO 대상
<code>glBufferData(...)</code>	72바이트 데이터가 CPU → GPU 메모리로 복사됨

VAO를 쓰지 않을 때

```
glBindBuffer(GL_ARRAY_BUFFER, vbo)
glVertexAttribPointer(0, ...) # 또 설명
glVertexAttribPointer(1, ...) # 또 설명
glEnableVertexAttribArray(0)
glEnableVertexAttribArray(1)
glDrawArrays(...)
```

VAO를 쓸 때

```
glBindVertexArray(vao) # “5번 설명서 꺼내!”
glDrawArrays(...)      # “그대로 그려!”
```

기본 코딩 – 시작 프로그램 (8/10)

`glEnableVertexAttribArray(0)` 0번 정점 속성(vertex attribute)을 이제부터 사용할 거야: 셰이더의 layout(location = 0)

`glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6*4, ctypes.c_void_p(0))`

`glEnableVertexAttribArray(1)`

`glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6*4, ctypes.c_void_p(3*4))`

`glBindVertexArray(0)` # VAO 바인딩 해제

`glUseProgram(shader)`

인자 순서	값	의미
1	0	이 설정은 0번 슬롯(location=0, aPos)에 적용해
2	3	한 정점당 3개 컴포넌트 (x,y,z)
3	GL_FLOAT	데이터 타입은 float
4	GL_FALSE	정규화 안 함 (0~1로 자동 변환 안 한다는 뜻. 색상일 때만 GL_TRUE 쓰는 경우 있음)
5	6*4 = 24	stride(스트라이드): 다음 정점이 시작되기까지 몇 바이트 떨어져 있냐? → 6개 float = 24바이트
6	ctypes.c_void_p(0)	offset(오프셋): 이 속성은 정점 시작에서부터 0바이트 뒤에서 시작해 (위치가 맨 앞이니까)

기본 코딩 – 시작 프로그램 (9/10)

고정된 뷰 & 투영 행렬 설정 (45도 시점에서 XYZ축 잘 보이게)

```
view = np.array([
    [ 0.7071, -0.4082, 0.5774, 0.0],
    [ 0.0000, 0.8165, 0.5774, 0.0],
    [-0.7071, -0.4082, 0.5774, 0.0],
    [ 0.0000, 0.0000, -3.4641, 1.0]
], dtype=np.float32)

proj = np.array([
    [ 1.7926, 0.0000, 0.0000, 0.0000],
    [ 0.0000, 2.4142, 0.0000, 0.0000],
    [ 0.0000, 0.0000, -1.0020, -1.0000],
    [ 0.0000, 0.0000, -0.2002, 0.0000]
], dtype=np.float32)
```

```
glUniformMatrix4fv(glGetUniformLocation(shader, "view"), 1, GL_FALSE, view)
glUniformMatrix4fv(glGetUniformLocation(shader, "proj"), 1, GL_FALSE, proj)
```

함수	사라졌나?	현재 상태 (OpenGL 4.6 / 2025년)	비고
glEnable(GL_LIGHTING)	사라짐	옛날 고정 파이프라인 → Core Profile 에서 완전 삭제	Legacy
glEnable(GL_TEXTURE_2D)	사라짐	같은 이유	Legacy
glEnable(GL_ALPHA_TEST)	사라짐	같은 이유	Legacy
glEnable(GL_DEPTH_TEST)	살아있음	완전 정식 지원, 지금도 매일 씀	현재 필수
glEnable(GL_BLEND)	살아있음	투명 처리에 필수	현재 필수
glEnable(GL_CULL)	살아있음	Cull face, stencil test 등도 사용	현재 필수
glEnableVertexArray()	살아있음	현대 OpenGL의 핵심 함수	현재 필수

glEnable(GL_DEPTH_TEST)

기본 코딩 – 시작 프로그램 (10/10)

```
def display(window):
    glClearColor(0.1, 0.1, 0.12, 1.0)
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    glBindVertexArray(vao)
    glDrawArrays(GL_TRIANGLES, 0, 3)

def reshape(window, w, h):
    glViewport(0, 0, w, h)

def keyboard(window, key, scancode, action, mods):
    if key == glfw.KEY_ESCAPE and action == glfw.PRESS:
        glfw.set_window_should_close(window, True)

# 메인
window = initialize_window(800, 600, "XYZ 삼각형 - R,G,B 꼭지점")
register_initGL(window, init_gl)
register_reshape(window, reshape)
register_keyboard(window, keyboard)
main_loop(window, display)
```



카메라 코드 개선 (강의 GitHub)

https://github.com/dknife/2025_2_Graphics/blob/main/Code/ModernGL/Camera.py

```
ModernGL > Camera.py > ...
1 # Camera.py (OpenGL 표준 lookAt 적용 버전)
2 import numpy as np
3 from OpenGL.GL import *
4 import math
5
6 def perspective(fov_y: float, aspect: float, near: float, far: float) -> np.ndarray:
7     """OpenGL 표준 perspective projection"""
8     f = 1.0 / math.tan(math.radians(fov_y) / 2.0)
9     nf = 1.0 / (near - far)
10    proj = np.array([
11        [f / aspect, 0.0, 0.0, 0.0],
12        [0.0, f, 0.0, 0.0],
13        [0.0, 0.0, (far + near) * nf, 2 * far * near * nf],
14        [0.0, 0.0, -1.0, 0.0]
15    ], dtype=np.float32)
16    return proj.T # GLSL column-major 맞춤
17
18 def look_at(eye, at, up) -> np.ndarray:
19     """OpenGL 표준 lookAt view matrix"""
20    eye = np.array(eye, dtype=np.float32)
21    at = np.array(at, dtype=np.float32)
22    up = np.array(up, dtype=np.float32)
23
24    f = at - eye
25    f /= np.linalg.norm(f) + 1e-8
26
27    r = np.cross(f, up)
28    r /= np.linalg.norm(r) + 1e-8
29
30    u = np.cross(r, f)
31
32    view = np.identity(4, dtype=np.float32)
33    view[0, :3] = r
34    view[1, :3] = u
35    view[2, :3] = -f
36    # translation 부분: -eye를 rotation으로 변환
37    view[:3, 3] = -np.dot(view[:3, :3], eye)
38
39    return view.T # GLSL column-major 맞춤
40
41 class Camera:
42     def __init__(self, eye=None, at=None, up=None):
```

```
camera = Camera()
camera.look_at([4.5, 2.5, 2.5], [0, 0, 0], [0, 1, 0])
```

```
def reshape(window, w, h):
    glViewport(0, 0, w, h)
    camera.set_aspect(w, h)
```

사용법

```
def display(window):
    glClearColor(0.1, 0.1, 0.12, 1.0)
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

```
camera.apply(shader) # 이 한 줄로 카메라 완전 적용!
```

```
glBindVertexArray(vao)
glDrawArrays(GL_TRIANGLES, 0, 3)
```


카메라 사용 코드 (강의 GitHub)

https://github.com/dknife/2025_2_Graphics/blob/main/Code/ModernGL/01_camera.py

```
ModernGL > Camera.py > ...
1 # Camera.py (OpenGL 표준 lookAt 적용 버전)
2 import numpy as np
3 from OpenGL.GL import *
4 import math
5
6 def perspective(fov_y: float, aspect: float, near: float, far: float) -> np.ndarray:
7     """OpenGL 표준 perspective projection"""
8     f = 1.0 / math.tan(math.radians(fov_y) / 2.0)
9     nf = 1.0 / (near - far)
10    proj = np.array([
11        [f / aspect, 0.0, 0.0, 0.0],
12        [0.0, f, 0.0, 0.0],
13        [0.0, 0.0, (far + near) * nf, 2 * far * near * nf],
14        [0.0, 0.0, -1.0, 0.0]
15    ], dtype=np.float32)
16    return proj.T # GLSL column-major 맞춤
17
18 def look_at(eye, at, up) -> np.ndarray:
19     """OpenGL 표준 lookAt view matrix"""
20    eye = np.array(eye, dtype=np.float32)
21    at = np.array(at, dtype=np.float32)
22    up = np.array(up, dtype=np.float32)
23
24    f = at - eye
25    f /= np.linalg.norm(f) + 1e-8
26
27    r = np.cross(f, up)
28    r /= np.linalg.norm(r) + 1e-8
29
30    u = np.cross(r, f)
31
32    view = np.identity(4, dtype=np.float32)
33    view[0, :3] = r
34    view[1, :3] = u
35    view[2, :3] = -f
36    # translation 부분: -eye를 rotation으로 변환
37    view[:3, 3] = -np.dot(view[:3, :3], eye)
38
39    return view.T # GLSL column-major 맞춤
40
41 class Camera:
42     def __init__(self, eye=None, at=None, up=None):
```

```
camera = Camera()
camera.look_at([4.5, 2.5, 2.5], [0, 0, 0], [0, 1, 0])
```

```
def reshape(window, w, h):
    glViewport(0, 0, w, h)
    camera.set_aspect(w, h)
```

사용법

```
def display(window):
    glClearColor(0.1, 0.1, 0.12, 1.0)
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

```
camera.apply(shader) # 이 한 줄로 카메라 완전 적용!
```

```
glBindVertexArray(vao)
glDrawArrays(GL_TRIANGLES, 0, 3)
```

카메라 사용 코드 (강의 GitHub)

https://github.com/dknife/2025_2_Graphics/blob/main/Code/ModernGL/01_camera.py

```
camera = Camera()
camera.look_at([4.5, 2.5, 2.5], [0, 0, 0], [0, 1, 0])

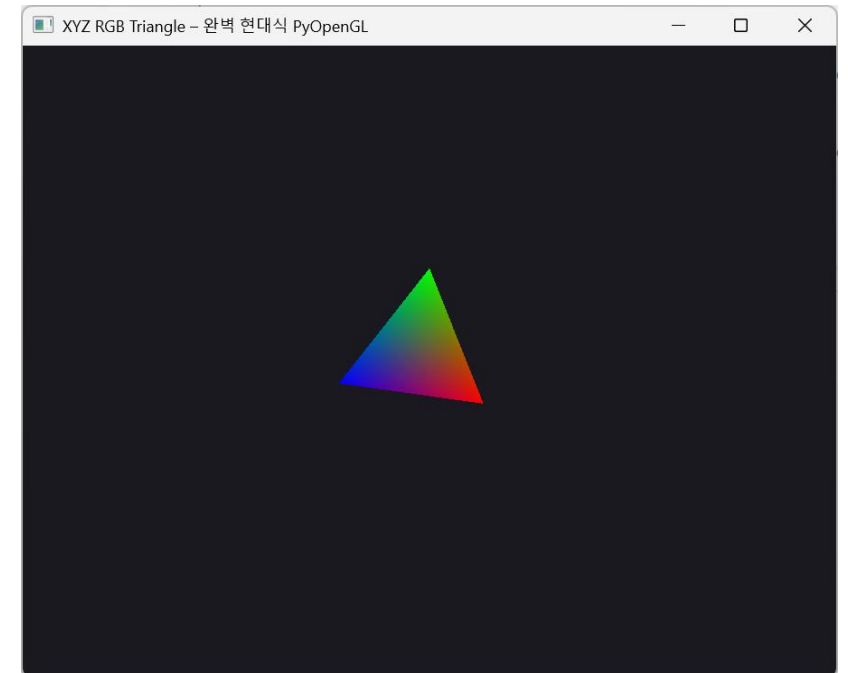
def reshape(window, w, h):
    glViewport(0, 0, w, h)
    camera.set_aspect(w, h)

def display(window):
    glClearColor(0.1, 0.1, 0.12, 1.0)
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

    camera.apply(shader)          # 이 한 줄로 카메라 완전 적용!

    glBindVertexArray(vao)
    glDrawArrays(GL_TRIANGLES, 0, 3)
```

사용 결과



Mesh

```
# ModernMesh.py
from OpenGL.GL import *
import numpy as np

class Mesh:
    def __init__(self):
        self.nV = 0
        self.nF = 0
        self.vertexBuffer = None
        self.normalBuffer = None
        self.idxBuffer = None

        self.vao = None
        self.vbo = None
        self.nbo = None
        self.ebo = None
```

법선 버퍼
엘리먼트 인덱스 버퍼

```

def loadMesh(self, filename):
    with open(filename, 'rt') as f:
        # 정점 읽기
        self.nV = int(next(f))
        self.vertexBuffer = np.zeros((self.nV, 3), dtype=np.float32)
        self.normalBuffer = np.zeros((self.nV, 3), dtype=np.float32)
        for i in range(self.nV):
            self.vertexBuffer[i] = np.array(next(f).split()[0:3], dtype=float)

        # 정규화
        coordMin = self.vertexBuffer.min()
        coordMax = self.vertexBuffer.max()
        scale = max(abs(coordMin), abs(coordMax))
        self.vertexBuffer /= scale

        # 면 읽기
        self.nF = int(next(f))
        self.idxBuffer = np.zeros((self.nF, 3), dtype=np.uint32)
        for i in range(self.nF):
            idx = np.array(next(f).split()[1:4], dtype=int)
            self.idxBuffer[i] = idx

            # 법선 계산 및 누적
            p0, p1, p2 = self.vertexBuffer[idx[0]], self.vertexBuffer[idx[1]], self.vertexBuffer[idx[2]]
            N = np.cross(p1 - p0, p2 - p0)
            self.normalBuffer[idx[0]] += N
            self.normalBuffer[idx[1]] += N
            self.normalBuffer[idx[2]] += N

        # 법선 정규화
        norms = np.linalg.norm(self.normalBuffer, axis=1)
        self.normalBuffer /= norms[:, np.newaxis]

```

메시 읽기: 바뀌지 않음

버퍼 준비

```
def setupGL(self):
    """VAO, VBO, EBO 생성 및 데이터 업로드"""
    self.vao = glGenVertexArrays(1)
    self.vbo = glGenBuffers(1)
    self.nbo = glGenBuffers(1)
    self.ebo = glGenBuffers(1)

    glBindVertexArray(self.vao)

    # Vertex
    glBindBuffer(GL_ARRAY_BUFFER, self.vbo)
    glBufferData(GL_ARRAY_BUFFER, self.vertexBuffer.nbytes, self.vertexBuffer, GL_STATIC_DRAW)
    glEnableVertexAttribArray(0)
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, None)

    # Normal
    glBindBuffer(GL_ARRAY_BUFFER, self.nbo)
    glBufferData(GL_ARRAY_BUFFER, self.normalBuffer.nbytes, self.normalBuffer, GL_STATIC_DRAW)
    glEnableVertexAttribArray(1)
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 0, None)

    # Indices
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, self.ebo)
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, self.idxBuffer.nbytes, self.idxBuffer, GL_STATIC_DRAW)

    glBindVertexArray(0)
```

그리기는 VAO 바인드 후에 실시

```
def draw(self):  
    glBindVertexArray(self.vao)  
    glDrawElements(GL_TRIANGLES, self.nF * 3, GL_UNSIGNED_INT, None)  
    glBindVertexArray(0)
```

Mesh 구현 (강의 github)

https://github.com/dknife/2025_2_Graphics/blob/main/Code/ModernGL/Mesh.py

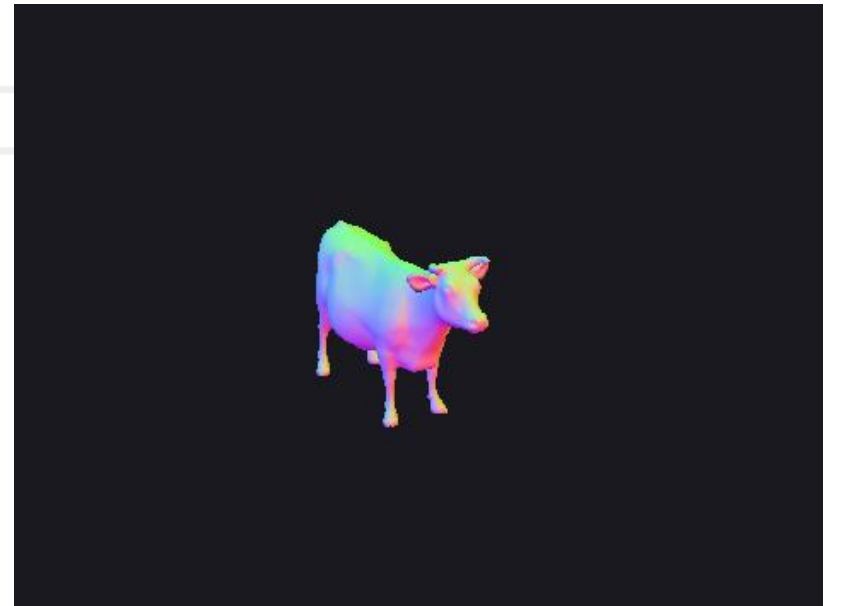
Mesh 로딩 예시

```
def init_gl(window):  
    global shader, camera, mesh  
  
    # 셰이더 컴파일  
    shader = compileProgram(  
        compileShader(vs, GL_VERTEX_SHADER),  
        compileShader(fs, GL_FRAGMENT_SHADER)  
    )  
    glUseProgram(shader)  
  
    # Mesh 로딩  
    mesh = Mesh()  
    mesh.loadMesh("./ModernGL/cow.txt")    # cow.txt 파일  
    mesh.setupGL()
```


Mesh 그리기 예시

https://github.com/dknife/2025_2_Graphics/blob/main/Code/ModernGL/02_mesh.py

```
def display(window):  
    glClearColor(0.1, 0.1, 0.12, 1.0)  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)  
  
    camera.apply(shader)  
    mesh.draw() # Mesh 렌더링
```



광원 처리는 모두 직접 구현

```
fs = ""
#version 460 core
struct Light {
    vec3 position;
    vec3 ambient;
    vec3 diffuse;
    vec3 specular;
    float shininess;
};
#define MAX_LIGHTS 8
uniform Light lights[MAX_LIGHTS];
uniform int lightActive[MAX_LIGHTS]; // struct 밖으로 이동

uniform vec3 eyePos; // ← 추가: 카메라 위치 전달

in vec3 FragPos;
in vec3 Normal;
out vec4 fragColor;

void main()
{
    vec3 norm = normalize(Normal);
    vec3 viewDir = normalize(eyePos - FragPos);
    vec3 result = vec3(0.0);

    for(int i=0; i<MAX_LIGHTS; i++){
        if(lightActive[i] == 0) continue; // struct 대신 별도 uniform 사용

        vec3 lightDir = normalize(lights[i].position - FragPos);
        float diff = max(dot(norm, lightDir), 0.0);
        vec3 reflectDir = reflect(-lightDir, norm);
        float spec = pow(max(dot(viewDir, reflectDir), 0.0), lights[i].shininess);

        vec3 ambient = lights[i].ambient;
        vec3 diffuse = lights[i].diffuse * diff;
        vec3 specular = lights[i].specular * spec;

        result += ambient + diffuse + specular;
    }

    fragColor = vec4(result, 1.0);
}
""
```

Light 클래스도 변경

https://github.com/dknife/2025_2_Graphics/blob/main/Code/ModernGL/Light.py

사용 예시

...

light = Light()

light.add_light(0, position=[1,2,3,1], diffuse=[1,1,1])

data = light.get_active_light_data()

glUniform3fv(glGetUniformLocation(shader, "lightPositions"), light.MAX_LIGHTS, data['positions'][:, :3])

glUniform3fv(glGetUniformLocation(shader, "lightDiffuse"), light.MAX_LIGHTS, data['diffuse'])

...

사용 예시 - 설정

```
# Light 설정
light = Light()
light.add_light(0, position=[2.0, 3.0, 2.0, 1.0], ambient=[0.1,0.1,0.1],
               diffuse=[1.0,1.0,0.0], specular=[1.0,1.0,1.0], shininess=120.0)

light.add_light(1, position=[-2.0, 1.0, 2.0, 1.0], ambient=[0.1,0.1,0.1],
               diffuse=[0.0,1.0,1.0], specular=[1.0,1.0,1.0], shininess=120.0)
```

사용 예시 - 적용

```
# Light uniform 전달
data = light.get_active_light_data()
for i in range(light.MAX_LIGHTS):
    prefix = f"lights[{i}]"
    glUniform3fv(glGetUniformLocation(shader, f"{prefix}.position"), 1, data['positions'][i][:3])
    glUniform3fv(glGetUniformLocation(shader, f"{prefix}.ambient"), 1, data['ambient'][i])
    glUniform3fv(glGetUniformLocation(shader, f"{prefix}.diffuse"), 1, data['diffuse'][i])
    glUniform3fv(glGetUniformLocation(shader, f"{prefix}.specular"), 1, data['specular'][i])
    glUniform1f(glGetUniformLocation(shader, f"{prefix}.shininess"), data['shininess'][i])
    glUniform1i(glGetUniformLocation(shader, "lightActive["+str(i)+"]"), int(data['active'][i]))

mesh.draw()
```

사용 코드

https://github.com/dknife/2025_2_Graphics/blob/main/Code/ModernGL/04_camerameshlight.py

